Looputo Finance — Security Review

Final Report

0xDING99YA

Independent smart-contract security researcher

August 12, 2026

Contents

Disclaimer **2**

About 0xDING99YA **2**

Risk Classification **2**

Protocol Summary **3**

Audit Details **3**

Scope . . . 3

Roles . . . 4

Executive Summary **4**

Issues found . . . 5

Remediation Status **6**

Findings **6**

Medium . . . 6

[M-01] Third-party Morpho repayment freezes pool collateral . . . 6

Low . . . 8

[L-01] External Morpho donations bypass the equity cap and opening fee . . . 8

[L-02] Unvalidated fee / LTV configuration can disable position operations . . . 9

[L-03] Revoking a core dependency as an aggregator clears its lending / flash-loan allowance 11

Disclaimer

I make every effort to surface as many vulnerabilities in the code as possible within the given time period, but hold no responsibility for the findings provided in this document. This security review is not an endorsement of the underlying business or product. The work was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts in scope.

This report represents a snapshot of the codebase at the time of review. Subsequent changes to the code — including those made in response to this report — fall outside the scope of this engagement unless explicitly contracted for re-review.

About 0xDING99YA

0xDING99YA is an independent smart-contract security researcher and a warden on Code4rena, where he has helped secure 18 protocols across competitive audits — including Ondo Finance, Panoptic, Arcade.xyz, Canto, Wildcat, Kelp DAO, reNFT, Rubicon and Dopex — with 25 High- and 30 Medium-severity findings validated to date.

His focus is DeFi lending, leverage and vault accounting — share-price manipulation, external-protocol integration assumptions and privileged-configuration surfaces — which is the profile of the Looputo codebase reviewed here.

Risk Classification

Severity is assessed using a standard impact × likelihood matrix:

		Impact		
		High	**Medium**	**Low**
Likelihood	**High**	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

- **Critical** — direct loss of user funds or permanent freezing of the protocol's primary functionality. Immediate remediation required.
- **High** — conditional loss of funds or permanent locking of user principal, reachable through a normal protocol lifecycle plus a permissionless trigger. Fix before launch.
- **Medium** — accounting drift, fairness violations, or stranded value gated by realistic preconditions (e.g. a semi-trusted role action), or destructive-but-unprofitable attacks whose cost is bounded and whose effect is recoverable by upgrade. Fix before launch.
- **Low** — defensive hardening, narrow attack windows, or issues bounded by stated trust assumptions.
- **Informational** — design observations, future-proofing concerns, and best-practice recommendations.

Protocol Summary

Looputo Finance is a protocol for automated leveraged positions. Users deposit assets into a shared pool and choose their own leverage; keepers watch the pool and step in to reduce liquidation and negative-carry risk.

Every pool keeps per-user collateral and debt share balances while supplying collateral and borrowing debt through one of three lending adapters — Aave V3, Spark or Morpho Blue. Position operations (open, add / reduce leverage, add / reduce equity, close) are executed atomically: each takes a Morpho flash loan and routes the swap leg through a DEX aggregator approved by the owner.

Pools are deployed as beacon proxies from an owner-controlled `Factory`. The initializer sets the collateral and debt assets, lending-market addresses, LTV limits, deposit cap, fees, approved aggregators, keeper and owner. A designated keeper can rebalance pools in batches via `deleverage` and `releverage`.

Audit Details

Field	Value
Auditor	0xDING99YA
Engagement type	Private security review (responsible disclosure)
Review period	July–August 2026
Language	Solidity ^0.8.20
Build framework	Foundry
Test status	All proofs-of-concept executed against a mainnet fork (block 25790000) of the in-scope code; pass
Remediation	Reviewed at commit 3a37647; 2 resolved, 2 acknowledged

Scope

The following contracts at commit `5c0e792` of `looputo-finance/Looputo-Finance` were the focus of this review:

Contract	Path
LoopPool	`src/LoopPool.sol`
MorphoLoopPool	`src/MorphoLoopPool.sol`
AaveV3LoopPool	`src/AaveV3LoopPool.sol`
SparkLoopPool	`src/SparkLoopPool.sol`
Factory	`src/Factory.sol`
LoopPoolBeacon	`src/LoopPoolBeacon.sol`
SharesLib / MathLib	`src/libraries/SharesLib.sol`, `src/libraries/MathLib.sol`

Out of scope:

- Morpho Blue, Aave V3 and Spark core contracts — treated as trusted dependencies, except where their documented on-chain behaviour is the enabler of an in-scope finding (notably the permissionless `onBehalf` parameter of `IMorpho.repay` / `supplyCollateral` in M-01 / L-01).
- OpenZeppelin libraries and vendored interface / library copies under `src/interfaces` and the `AaveV3`, `Spark` and `Morpho` sub-folders of `src/libraries`.
- DEX aggregators and their calldata; off-chain keeper, indexer and frontend components.

Roles

Role	Powers & trust assumption
Owner (Factory)	`setSystemTargetLTV`, `setPositionMaxLTV`, `setCap`, `setFeeConfiguration`, `setKeeper`, `setAggregator`, `setFlashloanFeePercentage`, beacon upgrade. Fully trusted (governance multisig).
Keeper	`deleverage` / `releverage` across user positions. Semi-trusted bot.
Fee collector	Receives open / close fees. Passive.
Permissionless	`openPosition`, `add/reducePositionLeverage`, `add/reducePositionEquity`, `closePosition`, `modifyPositionRateSpread`. No trust assumption — anyone may call. Also any account may call `IMorpho.repay` / `supplyCollateral` with the pool as `onBehalf`.

Executive Summary

This review identified 1 Medium and 3 Low findings across the in-scope contracts. Every finding was subsequently remediated or formally acknowledged by the team; see the Remediation Status section below.

The most consequential finding is structural and fully permissionless. `MorphoLoopPool` derives the pool's debt exclusively from the externally visible Morpho position, and Morpho allows anyone to repay debt or supply collateral on behalf of an arbitrary address. The headline findings follow from that one design decision:

- **M-01 (Medium)** — any account can repay the pool's entire Morpho debt on its behalf, driving external `borrowShares` to zero while internal `totalDebtShares` stays nonzero. From that state every close, add, reduce and keeper path reverts (zero/zero Morpho repay or division by zero in share minting), so all collateral of existing positions is frozen until the beacon implementation is upgraded. The attack is destructive rather than profitable — the attacker forfeits the repaid amount — which is why it is rated Medium rather than higher.
- **L-01 (Low)** — the same permissionless `supplyCollateral` / `repay` lets an existing share holder grow the pool position outside the `poolNetEquity` ledger, bypassing the equity cap and the opening fee.

The remaining findings address defensive hardening: missing range validation on fee and LTV configuration at initialization and in owner setters (L-02), and an allowance-slot collision that lets an owner mis-configuration wipe the core `MARKET` / `MORPHO` approvals (L-03). All Low findings were demonstrated with executable Foundry proof-of-concept tests against a mainnet fork; that exploit code is omitted from this status-of-record report and was delivered to the team separately.

Issues found

Severity	Count
Critical	0
High	0
Medium	1
Low	3
Info	0
Total	**4**

ID	Title	Severity	Status
M-01	Third-party Morpho repayment freezes pool collateral	Medium	Acknowledged
L-01	External Morpho donations bypass the equity cap and opening fee	Low	Resolved
L-02	Unvalidated fee / LTV configuration can disable position operations	Low	Resolved
L-03	Revoking a core dependency as an aggregator clears its lending / flash-loan allowance	Low	Acknowledged

Remediation Status

The in-scope contracts were remediated by the team in commit 3a37647. Each fix was independently re-reviewed against the updated source and re-verified by re-running the original proof-of-concept suite on a mainnet fork — the previously passing exploit tests now revert at the new validation, with no regressions in the surrounding logic.

Of the four findings, two are resolved and two are acknowledged with rationale: M-01 requires an attacker to burn capital equal to the pool's entire outstanding debt for no return, and the team accepts that risk given the beacon-upgrade recovery path; L-03 is bounded by the trust placed in the owner role, which is documented never to revoke core dependency addresses.

Disposition	Findings
Resolved (3a37647)	L-01, L-02
Acknowledged	M-01, L-03

This document is the status-of-record deliverable: exploit / proof-of-concept code has been omitted so the report is safe to distribute publicly. The complete executable PoCs were delivered to the team separately.

Findings

Medium

[M-01] Third-party Morpho repayment freezes pool collateral

Status: Acknowledged — economically irrational for the attacker. Executing the freeze requires repaying the pool's entire Morpho debt out of pocket with no way to recover it, and the outcome is recoverable through the beacon upgrade path. The team accepts the residual risk and will monitor the pool's external Morpho position for unexpected debt reductions.

Description

`MorphoLoopPool` treats the externally visible Morpho position as the sole source of truth for the pool's debt. `_cacheCollAndDebt` reads `position.borrowShares` and converts it to assets; every share-minting and share-burning path in `LoopPool` is driven by the delta of that value.

```
// MorphoLoopPool._cacheCollAndDebt
MorphoPosition memory position = IMorpho(MARKET).position(ID, address(this));
MorphoMarket memory market = IMorpho(MARKET).market(ID);
collAmount = uint256(position.collateral);
debtAmount = uint256(position.borrowShares).toAssetsUp(
    market.totalBorrowShares + VIRTUAL_SHARES,
    market.totalBorrowAssets + VIRTUAL_ASSETS
);
```

Morpho's `repay(marketParams, assets, shares, onBehalf, data)` has no authorization requirement on `onBehalf`. Any account can therefore repay the pool's full `borrowShares`, after which the

adapter reports `debtAmount == 0` while the inherited `totalDebtShares` and every user's `debtShares` remain unchanged. The adapter defines no reconciliation for this state, and `_repayDebt` has no zero-debt branch:

```
// MorphoLoopPool._repayDebt
uint256 shares = amounts.toSharesDown(...);
if (shares > uint256(position.borrowShares)) {
    repayInAmounts = false;
    shares = uint256(position.borrowShares);   // == 0 after the external repay
} else {
    repayInAmounts = true;
}
if (repayInAmounts) {
    IMorpho(MARKET).repay(MARKET_PARAMS, amounts, 0, address(this), "");
} else {
    IMorpho(MARKET).repay(MARKET_PARAMS, 0, shares, address(this), ""); // repay(0, 0) ->
    ↪ Morpho reverts
}
```

From the zero-external-debt state:

- **Close.** `closePosition` computes `userDebtAmount == 0`, requests a zero-value flash loan, and the reduce branch of `onMorphoFlashLoan` unconditionally calls `_repayDebt(0)`, which submits a `repay` with both `assets` and `shares` equal to zero. Morpho reverts on that input.
- **Debt-adding paths** (`openPosition`, `addPositionLeverage`, `addPositionEquity`, `releverage`). Share minting calls `SharesLib.toSharesUp(delta, totalAssets = 0, totalShares > 0)`, which divides by `totalAssets` and reverts.
- **Debt-reducing paths** (`reducePositionLeverage`, `reducePositionEquity`, `deleverage`). Repayment shares are capped to the external zero balance and the same zero/zero `repay` is submitted.

No existing user, new user, owner or keeper path can restore consistency or withdraw the stranded collateral through the contract's interface.

Impact

A well-funded party can pay off the pool's aggregate Morpho debt and freeze all collateral held for existing positions. The cost is bounded by the pool's outstanding debt, the attacker receives nothing back, and the position remains healthy on Morpho — so the only recovery is a beacon upgrade that introduces new accounting or a bespoke rescue function. Because the attack is destructive rather than profitable and the pool is upgradeable, this is rated **Medium** rather than Critical / High; it is nonetheless a permissionless, low-complexity denial of the protocol's primary function.

Recommended mitigation

Stop deriving share backing from the permissionlessly modifiable external position. Track collateral and borrow shares that changed *through the adapter* and read the pool's state from that accounting:

```
uint256 internal accountedCollateral;
uint256 internal accountedBorrowShares;
```

```
function _cacheCollAndDebt() internal view override returns (uint256 collAmount, uint256
    ↪ debtAmount) {
    MorphoMarket memory market = IMorpho(MARKET).market(ID);
    collAmount = accountedCollateral;
    debtAmount = accountedBorrowShares.toAssetsUp(
        market.totalBorrowShares + VIRTUAL_SHARES,
        market.totalBorrowAssets + VIRTUAL_ASSETS
    );
}

function _borrowDebt(uint256 amounts) internal override {
    (, uint256 sharesBorrowed) = IMorpho(MARKET).borrow(MARKET_PARAMS, amounts, 0, address(
    ↪ this), address(this));
    accountedBorrowShares += sharesBorrowed;
}

function _repayDebt(uint256 amounts) internal override {
    if (amounts == 0) return;
    ...
    (, uint256 sharesRepaid) = IMorpho(MARKET).repay(...);
    accountedBorrowShares -= sharesRepaid;
}
```

Route `_checkSystemLTV`, `_checkUserPositionLTV` and `_checkSystemAndUserPositionLTV` through `_cacheCollAndDebt()` so the LTV checks use the same view. Additionally, make `closePosition` skip the flash loan and withdraw collateral directly when `userDebtAmount == 0`, and return early from `_repayDebt` when the resolved `shares` is zero so a `repay(0, 0)` can never be submitted.

Does this fix the problem without opening a new one? Internal accounting closes both M-01 and L-01, but it changes what happens to value that arrives outside the adapter. After the patch an external repay reduces the pool's *real* Morpho debt below `accountedBorrowShares`; the next adapter-driven `repay` would then attempt to burn more shares than the position holds and revert for underflow. The remediation must therefore clamp `shares` to `min(accountedBorrowShares, position.borrowShares)` at repay time and treat the difference as a donation that lowers the debt-share price — or expose an owner-only `syncExternalPosition()` that credits any surplus to `accountedBorrowShares` explicitly. Either way, document that donated collateral / repaid debt is shared pro-rata by all share holders and is not attributable to the donor.

Low

[L-01] External Morpho donations bypass the equity cap and opening fee

Status: Resolved — commit 3a37647. `MorphoLoopPool` now tracks `accountedCollateral` / `accountedBorrowShar` updated only by the adapter hooks, and `_cacheCollAndDebt` and the LTV checks read from that accounting instead of `IMorpho.position`. Re-reviewed and confirmed: the PoC now fails at the assertion that the donor's shares own the donation.

Description

The same permissionless `onBehalf` surface as M-01 applies to `IMorpho.supplyCollateral`. Because `_cacheCollAndDebt` reads the external position, any collateral supplied or debt repaid directly on Morpho for the pool is immediately counted as backing for all internal shares — but `poolNetEquity`,

the cap and fee ledger, is only updated by the pool's own deposit entrypoints:

```
// LoopPool._increaseNetEquity — only reached from openPosition / addPositionEquity
function _increaseNetEquity(bool isColl, uint256 amount) internal {
    if (isColl) poolNetEquity += convertCollToDebt(amount);
    else        poolNetEquity += amount;
    require(poolNetEquity <= cap, "Deposit amount exceeds cap");
}
```

A user who is the sole (or dominant) share holder can therefore supply collateral to Morpho with `onBehalf = pool`, retain the full economic benefit through their unchanged shares, and later borrow, add leverage or close against the larger equity. The proof-of-concept opens a position just under the cap, shows a normal `addPositionEquity` being rejected by the cap, donates 1 wstETH directly on Morpho, and confirms the user's redeemable collateral grows by the full donation with no cap check, no opening fee and no change to `poolNetEquity`; `closePosition` then pays the donated equity out.

Impact

A position owner can grow protocol-managed equity and borrowing capacity beyond the configured cap while avoiding the opening fee, defeating a risk limit intended to bound pool exposure, and `poolNetEquity` diverges permanently from the value the Morpho position actually represents. Donated value is shared pro-rata, so there is no loss to other users; the trigger requires the donor to already hold shares and to give up the fee they would otherwise pay. Bounded to cap / fee circumvention, hence Low.

Recommended mitigation

Apply the adapter-level accounting described under M-01 (`accountedCollateral` / `accountedBorrowShares`) so that permissionless Morpho supplies or repayments no longer increase existing users' redeemable equity or borrowing capacity. With that change the PoC fails at the assertion that the donor's shares own the donation. If the team prefers to keep external reads, at minimum re-derive `poolNetEquity` from the live position at every cap check rather than from the ledger, and document that the opening fee can be avoided by donating.

[L-02] Unvalidated fee / LTV configuration can disable position operations

Status: Resolved — commit 3a37647. `_validateFeeConfiguration` and `_validateLTVConfiguration` are now enforced in `__LoopPool_init`, `setSystemTargetLTV`, `setPositionMaxLTV` and `setFeeConfiguration`, with an adapter-specific `_maximumLTV()`. Re-reviewed and confirmed: all three PoC deployments / setter calls now revert with `Fee exceeds maximum` or `Fee collector address cannot be zero address`.

Description

`__LoopPool_init` stores `systemTargetLTV`, `positionMaxLTV`, `openPositionFee`, `closePositionFee` and `feeCollector` without range or relationship checks, and the owner setters (`setSystemTargetLTV`, `setPositionMaxLTV`, `setFeeConfiguration`) only require the new value to differ from the current one (the fee setter additionally rejects a zero collector, but the initializer does not). Later arithmetic assumes the values are sane:

```
// LoopPool.openPosition
```

```
uint256 fee = params.amount.wMulDown(openPositionFee);
if (fee > 0) inToken.safeTransfer(feeCollector, fee);   // reverts for feeCollector == 0
_increaseNetEquity(params.isColl, params.amount - fee); // underflows for openPositionFee >
    ↪ WAD

// LoopPool.closePosition / reducePositionEquity
uint256 fee = withdrawableEquity.wMulDown(closePositionFee);
DEBT_TOKEN.safeTransfer(msg.sender, withdrawableEquity - fee); // underflows for
    ↪ closePositionFee > WAD
```

Concretely: an opening fee above `1e18` makes every `openPosition` / `addPositionEquity` revert; a closing fee above `1e18` makes every `closePosition` / `reducePositionEquity` revert; a nonzero fee with `feeCollector == address(0)` reverts on the ERC-20 transfer for standard tokens (wstETH included); a zero or inverted LTV pair makes every debt-bearing operation fail its post-operation LTV check, and a `positionMaxLTV` above the market LLTV lets positions be opened that Morpho will liquidate. The PoC deploys an `AaveV3LoopPool` through the real `Factory` with a 110% open fee and with a zero collector, and separately sets a 110% fee through `Factory.setFeeConfiguration`; in all three cases a user's `openPosition` reverts with no shares minted.

Impact

A deployment or owner update with these values disables opening, adding equity or closing positions for the whole pool until the owner submits a valid replacement configuration. The initializer accepts the values before any position exists, so the failure can be seeded at deployment rather than caught by a controlled operation. Requires owner error or compromise and is recoverable through the same setters, hence Low.

Recommended mitigation

Validate at initialization and in every setter: fees $\leq$ WAD; a nonzero collector whenever either fee is nonzero; `systemTargetLTV > 0`, `positionMaxLTV >= systemTargetLTV`, and `positionMaxLTV <= _maximumLTV()` where `_maximumLTV()` is adapter-specific (`MARKET_PARAMS.lltv` for Morpho and `1e4` BPS for Aave V3 / Spark):

```
function _validateFeeConfiguration(FeeConfiguration memory f) internal pure {
    require(f.openPositionFee <= WAD && f.closePositionFee <= WAD, "Fee exceeds maximum");
    require(f.feeCollector != address(0) || (f.openPositionFee == 0 && f.closePositionFee ==
    ↪ 0),
            "Fee collector address cannot be zero address");
}
function _validateLTVConfiguration(uint256 targetLTV, uint256 maxLTV) internal view {
    require(targetLTV > 0, "System target LTV cannot be zero");
    require(maxLTV >= targetLTV, "Position max LTV below system target LTV");
    require(maxLTV <= _maximumLTV(), "Position max LTV exceeds maximum");
}
```

In `MorphoLoopPool.initialize`, assign `MARKET_PARAMS` / `ID` *before* calling `__LoopPool_init` so `_maximumLTV()` can read the market LLTV. Mitigation safety: the checks are pure range guards and reject only configurations that already brick the pool; they introduce no new state or external call.

[L-03] Revoking a core dependency as an aggregator clears its lending / flash-loan allowance

Status: Acknowledged — trusted-role assumption. The owner is a governance multisig that is documented never to register `MARKET` or `MORPHO` as an aggregator, and the state is recoverable by re-approving the address. No contract change; recorded as an operational constraint on `setAggregator`.

Description

`__LoopPool_init` grants unlimited `COLL_TOKEN` / `DEBT_TOKEN` allowances to the core `MARKET` and `MORPHO` dependencies, then grants the same unlimited allowances to each configured aggregator. `setAggregator` uses the same ERC-20 spender slots and never checks that the address is not a core dependency:

```
// LoopPool.setAggregator (reached via Factory.setAggregator)
if (approved) {
    approvedAggregator[aggregator] = true;
    COLL_TOKEN.forceApprove(aggregator, type(uint256).max);
    DEBT_TOKEN.forceApprove(aggregator, type(uint256).max);
} else {
    approvedAggregator[aggregator] = false;
    COLL_TOKEN.forceApprove(aggregator, 0);   // also wipes MARKET / MORPHO allowance if
    ↪ aliased
    DEBT_TOKEN.forceApprove(aggregator, 0);
}
```

If the owner approves `MARKET` or `MORPHO` as an aggregator and later revokes it, the revocation zeroes the dependency approval. A `MARKET` alias breaks `_supplyCollateral` / `_repayDebt` in the Aave V3, Spark and Morpho adapters; a `MORPHO` alias removes the debt-token approval Morpho pulls to settle every flash loan, breaking all position operations regardless of adapter. The PoC demonstrates both sequences on a beacon-proxy `AaveV3LoopPool`: a control `openPosition` succeeds, the owner approves then revokes the core address through `Factory.setAggregator`, the corresponding allowances read zero, and a second user's `openPosition` reverts with no shares minted.

Impact

An owner configuration sequence leaves every open, leverage, equity, repayment or close operation reverting until a privileged action restores the approval; in the `MORPHO` case no position can be modified or closed. Requires an owner mistake and is recoverable (re-approving the alias restores the allowance), so Low.

Recommended mitigation

Reject core dependency addresses in both the initializer's aggregator loop and `setAggregator`:

```
require(aggregator != MARKET && aggregator != address(MORPHO), "Aggregator cannot be a core
    ↪ dependency");
```

Longer term, replace standing `type(uint256).max` aggregator allowances with a per-call approval inside `onMorphoFlashLoan` (approve exactly the swap input before the aggregator call, reset to zero afterwards) so revocation never touches shared allowance state and a compromised aggregator cannot drain idle balances. Mitigation safety: the guard only blocks two specific addresses that have no legitimate reason to be a swap router; per-call approvals add two `approve` calls per operation but remove

the standing-allowance exposure entirely.